```
import re

# ========================================
# 1. CORE UNIFICATION ALGORITHM
# ========================================

def is_variable(expr):
    return isinstance(expr, str) and expr.startswith('?')

def substitute(substitution, expr):
    if not substitution:
        return expr
    if is_variable(expr):
        return substitution.get(expr, expr)
    if isinstance(expr, list):
        return [substitute(substitution, arg) for arg in expr]
    return expr

def unify(x, y, theta=None):
    if theta is None:
        theta = {}
    x = substitute(theta, x)
    y = substitute(theta, y)

    if x == y:
        return theta
    if is_variable(x):
        return unify_var(x, y, theta)
    if is_variable(y):
        return unify_var(y, x, theta)
    if isinstance(x, list) and isinstance(y, list):
        if len(x) != len(y):
            return None
        for el_x, el_y in zip(x, y):
            theta = unify(el_x, el_y, theta)
            if theta is None:
                return None
        return theta
    return None

def unify_var(var, val, theta):
    if var == val:
        return theta
```

```
        return theta
    if isinstance(val, list) and var in val:
        return None
    new_theta = theta.copy()
    new_theta[var] = val
    return new_theta

# ========================================
# 2. OPTIMIZED INFERENCE ENGINE
# ========================================

class ForwardReasoner:
    def __init__(self):
        self.kb_facts = []          # List of unique facts (parsed format)
        self.kb_facts_set = set()   # Fast string lookup to prevent duplicate work
        self.rules = []             # List of rules

    def add_fact(self, fact):
        # Convert fact to an immutable string representation for fast verification
        fact_str = f"{fact[0]}({','.join(fact[1:])})"
        if fact_str not in self.kb_facts_set:
            self.kb_facts_set.add(fact_str)
            self.kb_facts.append(fact)
            return True
        return False

    def add_rule(self, rule_str):
        """Robust Parsing: IF cond1 & cond2 THEN result"""
        if "IF" not in rule_str or "THEN" not in rule_str:
            print("❌ Invalid rule structure! Must use clear 'IF ... THEN ...' syntax.")
            return

        # Isolate condition block and conclusion block accurately
        parts = rule_str.split("THEN")
        if_part = parts[0].replace("IF", "").strip()
        then_part = parts[1].strip()

        antecedents = [self._parse_predicate(cond) for cond in if_part.split("&") if cond.strip()]
        consequent = self._parse_predicate(then_part)

        self.rules.append({'antecedents': antecedents, 'consequent': consequent})

    def _parse_predicate(self, s):
        match = re.match(r"(\w+)\((.*)\)", s.strip())
        if match:
```

```
            pred = match.group(1)
            args = [arg.strip() for arg in match.group(2).split(",") if arg.strip()]
            return [pred] + args
        return [s.strip()]

    def match_conditions(self, antecedents, theta=None):
        if theta is None:
            theta = {}
        if not antecedents:
            return [theta]

        first = antecedents[0]
        rest = antecedents[1:]
        successful_thetas = []

        for fact in self.kb_facts:
            # Quick structural check before deep unification optimization
            if fact[0] != first[0] or len(fact) != len(first):
                continue

            match_theta = unify(first, fact, theta.copy())
            if match_theta is not None:
                results = self.match_conditions(rest, match_theta)
                successful_thetas.extend(results)

        return successful_thetas

    def run_inference(self):
        """Runs fast incremental forward checking loops."""
        print("\n⚡ Starting Highly Optimized Forward Reasoning Engine...")
        step = 1

        # Track rule combinations already executed to prevent infinite loops
        processed_combinations = set()

        while True:
            inferences_this_round = False

            for rule in self.rules:
                bindings_list = self.match_conditions(rule['antecedents'])

                for bindings in bindings_list:
                    # Create an immutable snapshot key of this binding application
                    binding_key = (id(rule), tuple(sorted(bindings.items())))
                    if binding_key in processed_combinations:
```

```
                    continue

                processed_combinations.add(binding_key)
                new_fact = substitute(bindings, rule['consequent'])

                if self.add_fact(new_fact):
                    readable_fact = f"{new_fact[0]}({', '.join(new_fact[1:])})"
                    print(f"✅ [Step {step}] Derived: {readable_fact}")
                    inferences_this_round = True
                    step += 1

            # Break out if no new updates can be made
            if not inferences_this_round:
                break

        print("\n🎯 Complete! Final Knowledge Base facts list:")
        for fact_str in sorted(self.kb_facts_set):
            print(f" ➔ {fact_str}")


# ========================================
# 3. INTERACTIVE CONSOLE EXECUTION
# ========================================

if __name__ == "__main__":
    reasoner = ForwardReasoner()

    print("======================================================")
    print("   High-Performance Forward Chaining Inference System")
    print("======================================================\n")

    print("--- STEP 1: Enter Inference Rules (Type 'done' when complete) ---")
    print("Format -> IF human(?x) THEN mortal(?x)")
    print("Format -> IF parent(?x, ?y) & male(?x) THEN father(?x, ?y)\n")

    while True:
        r_input = input("Add Rule: ").strip()
        if r_input.lower() == 'done':
            break
        if r_input:
            reasoner.add_rule(r_input)

    print("\n--- STEP 2: Enter Base Facts (Type 'done' when complete) ---")
    print("Format -> human(Socrates)")
```

```
    print("Format -> parent(John, Mary)")
    print("Format -> male(John)\n")

    while True:
        f_input = input("Add Fact: ").strip()
        if f_input.lower() == 'done':
            break
        if f_input:
            parsed_fact = reasoner._parse_predicate(f_input)
            reasoner.add_fact(parsed_fact)

    reasoner.run_inference()
```

```
==================================================
   High-Performance Forward Chaining Inference System
==================================================

--- STEP 1: Enter Inference Rules (Type 'done' when complete) ---
Format -> IF human(?x) THEN mortal(?x)
Format -> IF parent(?x, ?y) & male(?x) THEN father(?x, ?y)

Add Rule: IF CartTotal(?c, ?amount) AND (?amount >= 100) THEN EligibleForFreeShipping(?c)
Add Rule: IF MemberLevel(?c, Gold) THEN ApplyDiscount(?c, 0.15)
Add Rule: IF EligibleForFreeShipping(?c) AND ApplyDiscount(?c, ?disc) THEN SendFreeGift(?c)
Add Rule: done

--- STEP 2: Enter Base Facts (Type 'done' when complete) ---
Format -> human(Socrates)
Format -> parent(John, Mary)
Format -> male(John)

Add Fact: Customer(Alice)
Add Fact: MemberLevel(Alice, Gold)
Add Fact: CartTotal(Alice, 120)
Add Fact: FirstTimeBuyer(Alice, False)
Add Fact: done

⚡ Starting Highly Optimized Forward Reasoning Engine...
✅ [Step 1] Derived: EligibleForFreeShipping(Alice)
✅ [Step 2] Derived: ApplyDiscount(Alice, 0.15)

🎯 Complete! Final Knowledge Base facts list:
 ➔ ApplyDiscount(Alice,0.15)
 ➔ CartTotal(Alice,120)
 ➔ Customer(Alice)
 ➔ EligibleForFreeShipping(Alice)
```

```
➔ FirstTimeBuyer(Alice,False)
➔ MemberLevel(Alice,Gold)
```